A

Real-time Research Project Report
on

REAL TIME HUMAN EMOTION DETECTION

Submitted for partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
in

COMPUTER SCIENCE AND ENGINEERING

by

S. NAGA JYOTHI - 24K81A05J0

S. SAMVIDHA - 24K81A05H9

C. SIDDHARTH RAO - 24K81A05H8

Under the Guidance of

Mrs.SHAZIYA NUSRATH.SK

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous
Affiliated to JNTUH, Approved by AICTE
Accredited by NBA & NAAC A+, ISO 9001-2008 Certified
Dhulapally, Secunderabad-500 100
www.smec.ac.in

JUNE - 2026

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001:2008 Certified

Dhulapally, Secunderabad - 500100

www.smec.ac.in

CERTIFICATE

This is to certify that the project entitled **"REAL TIME HUMAN EMOTION DETECTION"** is being submitted **by S. NAGA JYOTHI (24K8105J0), S. SAMVIDHA (24K81A05H9), C. SIDDHARTH (24K81A05H8)** in fulfilment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING** is recorded of bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Internal Guide
Mrs.SHAZIYA NUSRATH.SK
Assistant Professor
Department of CSE

Head of the Department
Mr. D. Krishna Kishore
Associate Professor & HOD
Department of CSE

Place: Dhulapally, Secunderabad
Date:

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001:2008 Certified

Dhulapally, Secunderabad - 500100

www.smec.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, the students of '**Bachelor of Technology in Department of computer science and engineering'**, session: 2024-2028**, St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad,** hereby declare that the work presented in this project work entitled **"REAL TIME HUMAN EMOTION DETECTION"** is the outcome of our own bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

S. NAGA JYOTHI - 24K81A05J0
S. SAMVIDHA - 24K81A05H9
C. SIDDHARTH RAO - 24K81A05H8

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, we would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

We especially would like to express our deep sense of gratitude and indebtedness to **Prof.Dr.K.RAVINDRA**, Director & Principal, St. Martin's Engineering collage Dhulapally, for permitting us to undertake this project.

We also thankful to, **Mr. D. KRISHNA KISHORE,** Head of the Department, Computer science and engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project as well as Project Coordinator **Mr. M. Dileep Kumar**, Assistant Professor, Department of Computer science and Engineering for his valuable support.

We would like to express our sincere gratitude and indebtedness to our project supervisor **Mrs.SHAZIYA NUSRATH.SK**, Assistant Professor, Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, for his support and guidance throughout our project.

Finally, we express thanks to all those who have helped us successfully completing this project. Furthermore, we would like to thank our family and friends for their moral support and encouragement. we express thanks to all those who have helped us in successfully completing the project.

S. NAGA JYOTHI - 24K81A05J0
S. SAMVIDHA - 24K81A05H9
C. SIDDHARTH RAO - 24K81A05H8

ABSTRACT

Human emotion recognition plays an important role in improving communication between humans and computers. This project presents a Real-Time Human Emotion Detection System that identifies a person's emotions by analyzing both facial expressions and voice signals. Facial emotions are detected using a webcam with a Convolutional Neural Network (CNN)**,** while voice emotions are recognized by extracting Mel Frequency Cepstral Coefficients (MFCC) features and classifying them using a Neural Network**.** The outputs from both modalities are combined using an emotion fusion technique to improve the accuracy and reliability of the final prediction. To ensure stable results, the system applies face smoothing, history smoothing, majority voting, and stable print gating, which reduce sudden fluctuations in emotion detection. Voice processing runs in a separate thread, allowing simultaneous face and voice analysis without affecting real-time performance. The detected face emotion, voice emotion, and final emotion are displayed on a live webcam screen through a Heads-Up Display (HUD), and the final emotion can also be announced using a text-to-speech module. This multimodal approach provides more accurate and robust emotion recognition than systems that rely on only facial expressions or voice, making it suitable for applications in healthcare, education, human-computer interaction,surveillance,and intelligent virtual assistants.

UGC AUTONOMOUS

CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
CHAPTER 1 INTRODUCTION	1
1.1 Objective	1
1.2 Overview	1
CHAPTER 2 LITERATURE SURVEY	2
CHAPTER 3 SYSTEM ANALYSIS AND DESIGN	4
3.1 Existing System	4
3.2 Proposed System	5
3.3 Algorithm	6
3.4 Input design	7
3.5 Output design	7
CHAPTER 4 SYSTEM REQUIREMENTS & SPECIFICATIONS	8
4.1 Hardware Requirements	8
4.2 software requirements	8
4.3 UML Diagrams	9
4.3.1 Use Case Diagram	10
4.3.2 Class Diagram	10
4.3.3 Sequence Diagram	11
4.3.4 Activity Diagram	12

4.4 Testing 14
4.4.1 Unit Testing 14
4.4.2 Integration Testing 14
4.4.3 System testing 15
4.4.4 Functional Testing 15
4.4.5 White Box Testing 15
4.4.6 Black Box Testing 16
4.4.7 Performance Testing 16
4.4.8 User Interface (UI) Testing 16
4.4.9 Security Testing 17
CHAPTER 5 SOURCE CODE 18
CHAPTER 6 EXPERIMENTAL RESULTS 26
CHAPTER 7 CONCLUSION & FUTURE ENHANCEMENT
7.1 CONCLUSION 27
7.2 FUTURE ENHANCEMENT 28
CHAPTER 8 REFERENCES 29

CHAPTER 1

INTRODUCTION

Human emotions are an essential part of communication and play a significant role in understanding human behavior. Recognizing emotions accurately helps improve the interaction between humans and intelligent computer systems. Traditional emotion recognition systems often rely only on facial expressions or speech, which may produce inaccurate results due to lighting variations, facial occlusions, background noise, or differences in speaking style. To overcome these limitations, multimodal emotion recognition combines information from multiple sources to improve accuracy and reliability.The Real-Time Human Emotion Detection System is designed to recognize emotions by analyzing both facial expressions and voice signals simultaneously. The system captures facial images through a webcam and identifies facial emotions using

a Convolutional Neural Network (CNN)**.** It also records speech through a microphone, extracts Mel Frequency Cepstral Coefficients (MFCC) features, and classifies voice emotions using a Neural Network. The face and voice predictions are combined using an emotion fusion technique to generate the final emotion. To provide stable and consistent results, the system applies face smoothing, history smoothing, majority voting, and stable print gating. Voice processing is performed in a separate thread to ensure smooth real-time execution. The detected emotions are displayed through a Heads-Up Display (HUD) on the live webcam screen, and the final emotion can also be announced using a text-to-speech module.

1.1 Objective

The main objective of this project is to develop a real-time human emotion detection system that accurately recognizes emotions by combining facial expressions and voice signals. The system aims to improve the accuracy of emotion recognition through multimodal analysis, reduce incorrect predictions using smoothing techniques, provide real-time emotion detection, and display the detected face emotion, voice emotion, and final emotion on a live webcam screen. The project also aims to create a reliable and efficient system that can be used in applications such as healthcare, education, human-computer interaction, surveillance, and virtual assistants.

1.2 Overview

The proposed system captures real-time video from a webcam and audio from a microphone. Facial emotions are detected using a CNN model, while voice emotions are recognized by extracting MFCC features and classifying them using a Neural Network. Both predictions are combined through an emotion fusion process to determine the final emotion.

CHAPTER 2

LITERATURE SURVEY

Recent research has shown significant advancements in the field of real-time human emotion recognition through the application of deep learning techniques. Aritra Banerjee et al. (2025) proposed a Hybrid Convolutional Neural Network (CNN) and Mixture-of-Experts (MoE) deep learning model for real-time facial emotion detection. Their approach was evaluated using the AffectNet and RAF-DB datasets, which contain a large variety of facial expression images captured under different conditions. The proposed hybrid architecture effectively combined both global and local facial features, enabling more accurate emotion recognition compared to conventional CNN models. Their work demonstrated that integrating multiple expert networks improves the system's ability to recognize complex facial expressions while increasing the robustness and overall performance of emotion detection systems. This research highlighted the importance of advanced deep learning architectures for improving facial emotion recognition accuracy in real-world applications.Another important contribution was made by Mayank Lahariya et al. (2021)**,** who developed a Facial Emotion Recognition system using an Ensemble CNN model. Their model was trained and tested on the widely used FER-2013 dataset, which contains thousands of facial images representing various human emotions. The ensemble learning approach combined multiple CNN models to improve classification performance and reduce prediction errors. The proposed system successfully recognized seven basic human emotions, namely happy, sad, angry, surprise, fear, disgust, and neutral**,** from images captured through a webcam. Their research demonstrated that combining multiple deep learning models can significantly enhance the reliability and accuracy of emotion recognition systems while making them suitable for real-time applications. The study also emphasized the effectiveness of deep learning in extracting meaningful facial features for accurate emotion classification.Similarly, Sergio Arriaga et al. (2017) introduced a Convolutional Neural Network (CNN) based on the Mini-Xception architecture for simultaneous emotion and gender classification. Their model was also trained using the FER-2013 dataset and achieved high accuracy while maintaining low computational complexity, making it suitable for real-time deployment. The Mini-Xception architecture reduced the number of model parameters without compromising prediction performance, allowing faster processing and efficient facial emotion detection.

Their work proved that lightweight CNN architectures could deliver accurate results while meeting the real-time requirements of practical emotion recognition systems. This research has become one of the most widely adopted approaches for facial emotion recognition because of its balance between computational efficiency and classification accuracy.Based on the findings of these studies, it can be concluded that deep learning techniques, particularly CNN-based architectures, have significantly improved the performance of emotion recognition systems. Hybrid models, ensemble learning, and lightweight CNN architectures each contribute to increasing recognition accuracy, robustness, and real-time performance. However, most existing systems primarily rely on facial expressions alone, which may lead to incorrect predictions when facial information is unclear or affected by environmental conditions. To overcome these limitations, the proposed project adopts a multimodal approach by combining facial expression analysis using CNN with voice emotion recognition using MFCC feature extraction and a Neural Network. The integration of face and voice emotions, together with smoothing techniques, majority voting, threading, and real-time Heads-Up Display (HUD), provides a more stable, reliable, and accurate emotion recognition system suitable for applications such as healthcare, education, human-computer interaction, surveillance, and intelligent virtual.

UGC AUTONOMOUS

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 EXISTING SYSTEM

Existing human emotion detection systems primarily rely on a single modality, such as facial expressions or voice signals, to recognize human emotions. Facial emotion recognition systems generally use cameras to capture facial expressions and apply image processing or deep learning techniques to classify emotions such as happy, sad, angry, surprise, fear, disgust, and neutral. However, these systems are highly dependent on factors such as lighting conditions, facial pose, occlusions, and image quality, which often reduce recognition accuracy. Similarly, voice-based emotion recognition systems analyze speech characteristics to identify emotions, but their performance is affected by background noise, microphone quality, speech variations, and differences in accent or speaking style.

Most existing systems do not combine facial and voice information, making them less reliable when one source of data is unclear or unavailable. They also produce unstable outputs because emotion predictions may change rapidly from one frame to another, resulting in inconsistent and inaccurate emotion detection. Many systems lack effective smoothing techniques, majority voting mechanisms, and fusion strategies to improve prediction stability. Furthermore, several existing approaches are not optimized for real-time applications, as they process face and voice separately instead of simultaneously. They often do not support multithreading, leading to slower execution and reduced system performance. In addition, most traditional emotion detection systems provide limited real-time visualization and do not include user-friendly interfaces such as live emotion display or voice feedback.

These challenges highlight the need for a multimodal real-time emotion detection system that combines facial expressions and voice signals while applying deep learning techniques, emotion fusion, smoothing algorithms, and real-time visualization to improve overall accuracy and performance.

3.2 PROPOSED SYSTEM

The proposed system is a Real-Time Human Emotion Detection System that accurately recognizes human emotions by combining facial expressions and voice signals using deep learning techniques. Unlike existing systems that rely on a single source of information, the proposed system performs multimodal emotion recognition, where emotions detected from both the face and voice are fused to produce a more reliable and accurate final prediction. The system captures facial expressions through a webcam and detects emotions using a Convolutional Neural Network (CNN) model. Simultaneously, it records the user's speech through a microphone, extracts Mel Frequency Cepstral Coefficients (MFCC) features using Librosa, and classifies voice emotions using a Neural Network.To improve the stability and consistency of the detected emotions, the proposed system applies face smoothing**,** history smoothing**,** majority voting**,** and stable print gating techniques. These methods reduce sudden fluctuations in emotion predictions and ensure that only stable emotions are displayed as the final output. The voice recording process runs in a separate thread, allowing face and voice emotion detection to execute simultaneously without interrupting the real-time webcam processing. A threading lock is used to safely share emotion data between threads and prevent synchronization issues.

The detected Face Emotion**,** Voice Emotion, and Final Emotion are displayed in real time through a color-coded Heads-Up Display (HUD) on the webcam screen. Additionally, the final detected emotion can be announced using a Text-to-Speech (TTS) module, making the system more interactive and user-friendly. The integration of multimodal emotion recognition, deep learning algorithms, smoothing techniques, multithreading, and real-time visualization significantly improves the accuracy, robustness, and efficiency of emotion detection. The proposed system is suitable for various real-world applications, including healthcare, education, human-computer interaction, mental health monitoring, customer service, surveillance, and intelligent virtual assistants, where reliable emotion recognition plays an important role.

3.3 ALGORITHM

Step 1: Start the system and initialize the webcam, microphone, deep learning models, and required libraries.

Step 2: Load the trained CNN model for facial emotion recognition and the Neural Network model for voice emotion recognition.

Step 3: Capture video frames continuously from the webcam. Step 4: Detect the face in each video frame using OpenCV.

Step 5: Preprocess the detected face (resize, normalize, and convert into the required input format).

Step 6: Predict the facial emotion using the CNN model.

Step 7: Simultaneously record the user's voice through the microphone using a separate thread.

Step 8: Extract MFCC (Mel Frequency Cepstral Coefficients) features from the recorded speech using Librosa.

Step 9: Predict the voice emotion using the trained Neural Network model.

Step 10: Combine the face emotion and voice emotion using the emotion fusion technique to obtain the final emotion.

Step 11: Apply face smoothing, history smoothing, and majority voting to improve the stability of the detected emotion.

Step 12: Apply stable print gating so that the final emotion is displayed only after consistent detection.

Step 13: Display the Face Emotion**,** Voice Emotion**,** and Final Emotion on the live webcam screen using a color-coded Heads-Up Display (HUD).

Step 14: Optionally announce the final detected emotion using the Text-to-Speech (TTS) module. **Step 15:** Repeat the above steps until the user exits the application.

Step 16: Release the webcam and microphone resources, close all windows, and terminate the program.

3.3 INPUT DESIGN

Input design defines the data that is provided to the system for processing. In the proposed Real-Time Human Emotion Detection System, the inputs are obtained from two different sources: the webcam and the microphone. The webcam continuously captures live video frames containing the user's facial expressions, while the microphone records the user's speech in real time. Before emotion recognition, the captured face images are preprocessed by detecting the face, resizing the image, and normalizing the pixel values. Similarly, the recorded speech is processed by extracting Mel Frequency Cepstral Coefficients (MFCC) features using the Librosa library. These preprocessed face images and voice features are then provided as inputs to the trained deep learning models for emotion prediction. The system is designed to accept real-time inputs continuously, enabling accurate and uninterrupted emotion detection.

3.4 OUTPUT DESIGN

Output design describes how the processed information is presented to the user. After analyzing both facial expressions and voice signals, the system predicts the face emotion and voice emotion separately using deep learning models. These predictions are then combined using an emotion fusion technique to generate the final emotion. To improve stability, the system applies face smoothing, history smoothing, majority voting, and stable print gating before displaying the final result. The detected Face Emotion, Voice Emotion, and Final Emotion are displayed in real time on the webcam screen through a color-coded Heads-Up Display (HUD). Additionally, the final detected emotion can be announced using the Text-to-Speech (TTS) module. The output is updated continuously, providing users with accurate, stable, and real-time emotion recognition.

UGC AUTONOMOUS

CHAPT ER 4

SYSTEM REQUIREMENTS & SPECIFICATIONS

4.1 HARDWARE REQUIREMENTS

1. **Processor** :Intel Core i5 (8th Generation or above)
2. **RAM** :Minimum 8 GB (16 GB recommended)
3. **Storage** :Minimum 256 GB SSD or 500 GB HDD
4. **Webcam** :Built-in or External HD Webcam
5. **Microphone** :Built-in or External Microphone
6. **Display** :14-inch or larger monitor
7. **Graphics** : Integrated Graphics (Dedicated GPU optional for faster processing)

4.2 SOFTWARE REQUIREMENTS:

1. **Operating System** : Windows 10 / Windows 11
2. **Programming Language** : Python 3.10 or above
3. **IDE** : Visual Studio Code (VS Code)
4. **OpenCV (cv2)** : Face detection and image processing
5. **TensorFlow** : Deep learning framework
6. **Keras** : Loading and running trained models
7. **NumPy** : Numerical computations and array processing
8. **Librosa :** Audio processing and MFCC feature extraction

ST. MARTIN'S ENGINEERING COLLEGE SMEC TECHNOLOGY FOR PROSPERITY UGC AUTONOMOUS

UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.It is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

4.2.1 USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

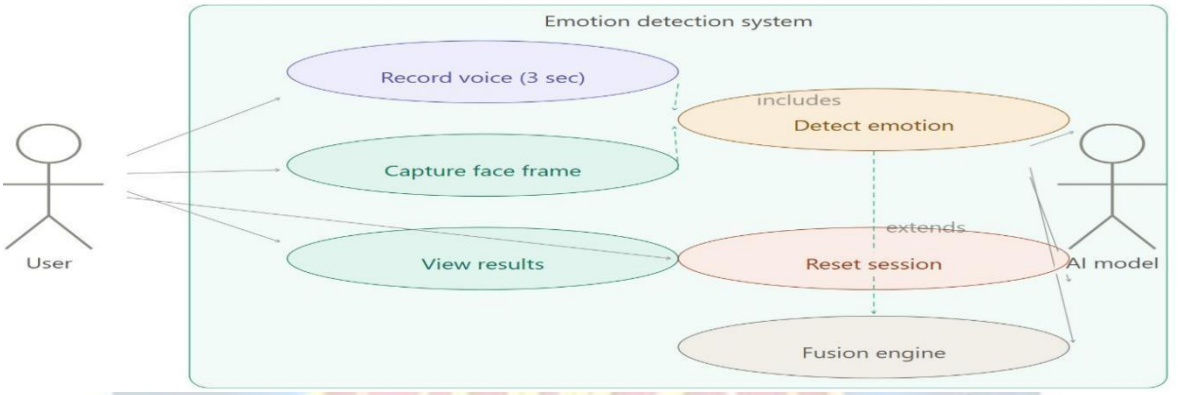


Figure 4.2.1: Use case Diagram

4.2.2 Class Diagram:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

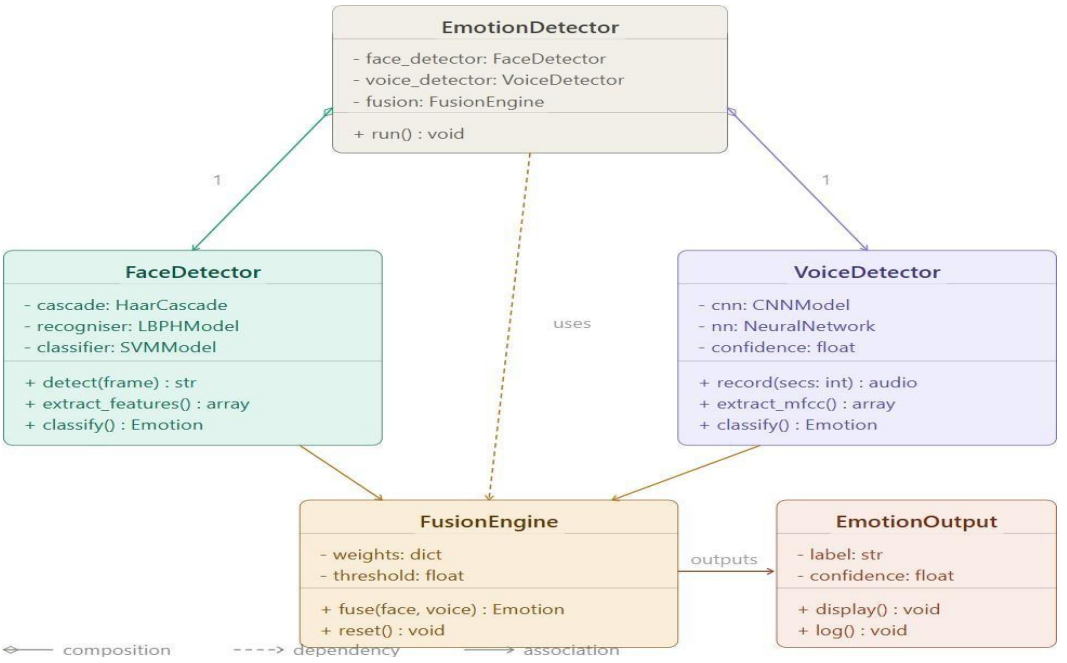


Figure 4.2.2 Class Diagram

4.2.3 Sequence Diagram:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios.

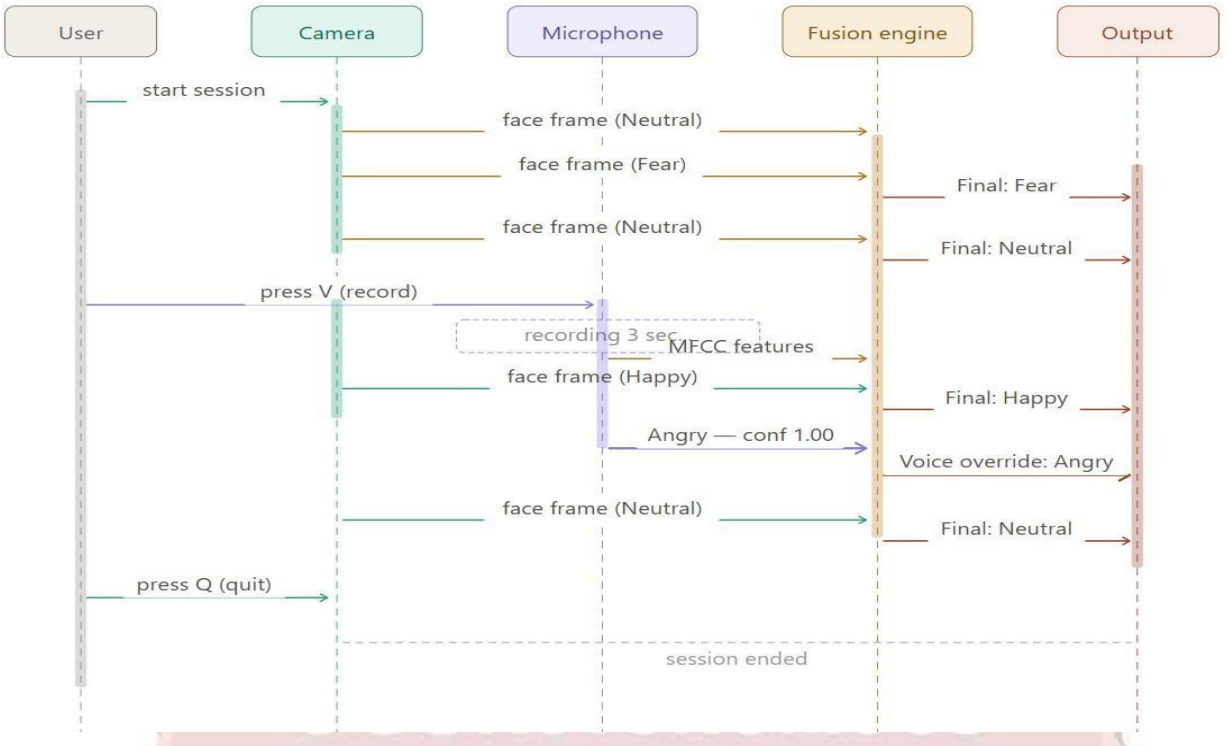


Figure 4.2.3: Sequence Diagram

4.2.4 ACTIVITY DIAGRAM:

The activity diagram for the real-time human emotion detection system illustrates the complete behavioral flow of the application from the moment the session starts to the point it ends. The diagram is divided into two swim lanes — the left teal lane represents the face detection pipeline that runs continuously in a loop, and the right purple lane represents the voice detection pipeline that is triggered on demand by the user pressing the V key. When the session begins, a fork bar splits the execution into two parallel threads, allowing both pipelines to operate simultaneously and independently of each other. In the face pipeline, the system first captures a frame from the webcam using OpenCV. That frame is then passed through the HaarCascade classifier, which scans the image to detect a face region. A decision diamond checks whether a face was actually found — if no face is detected, the system loops back and captures the next frame. When a face is found, the cropped region is passed to the LBPH recogniser, which extracts local binary pattern texture features from the face. Those features are then fed into the SVM classifier, which maps them to one of seven emotion labels — Happy, Fear, Angry, Sad, Surprise, Disgust, or Neutral. In your live session, this pipeline produced Neutral, then Fear, then Neutral again, then Happy, and finally Neutral — showing how quickly face emotions can change across frames.

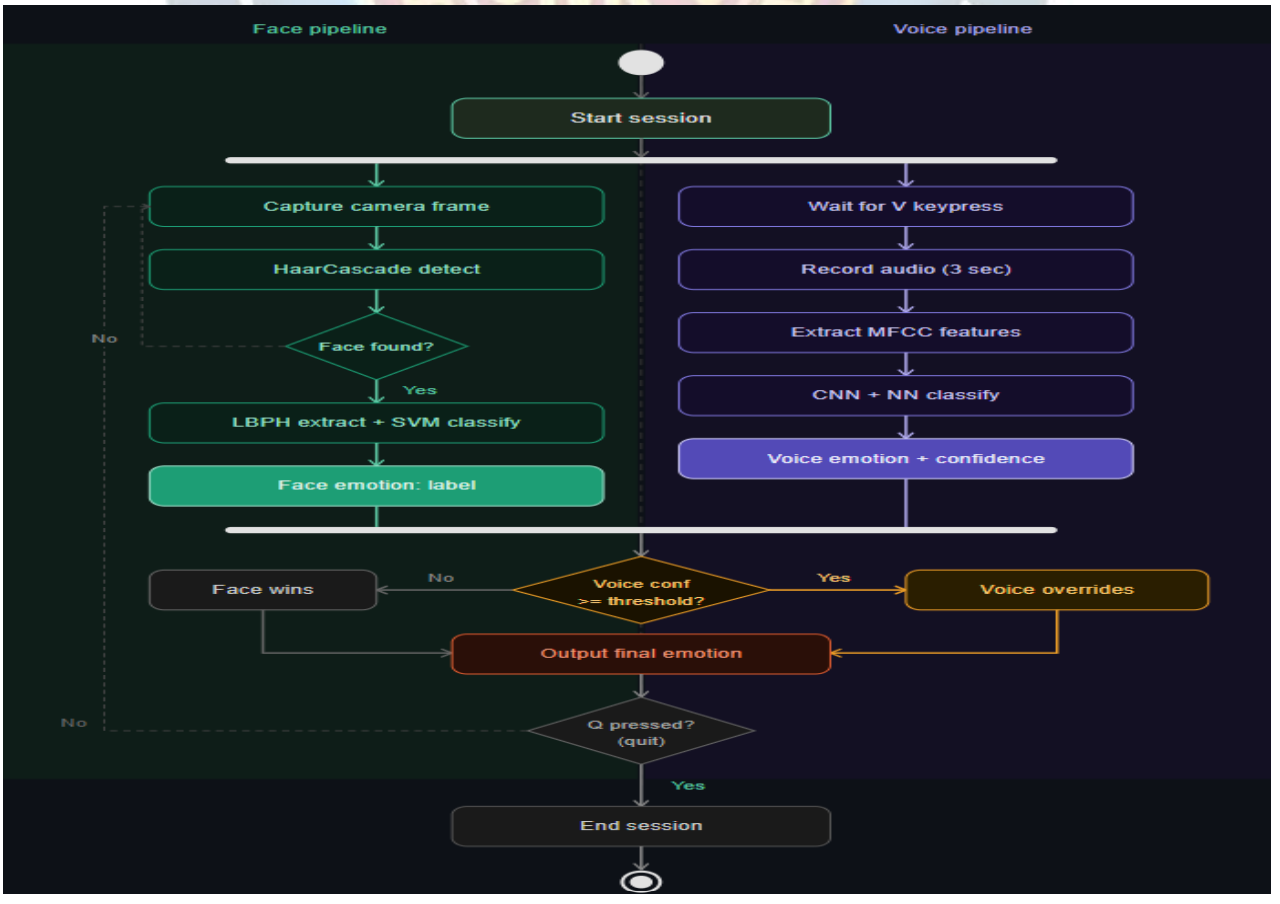


Figure 4.2.4: Activity Diagram

4.2.5 : SYSTEM ARCHITECTURE

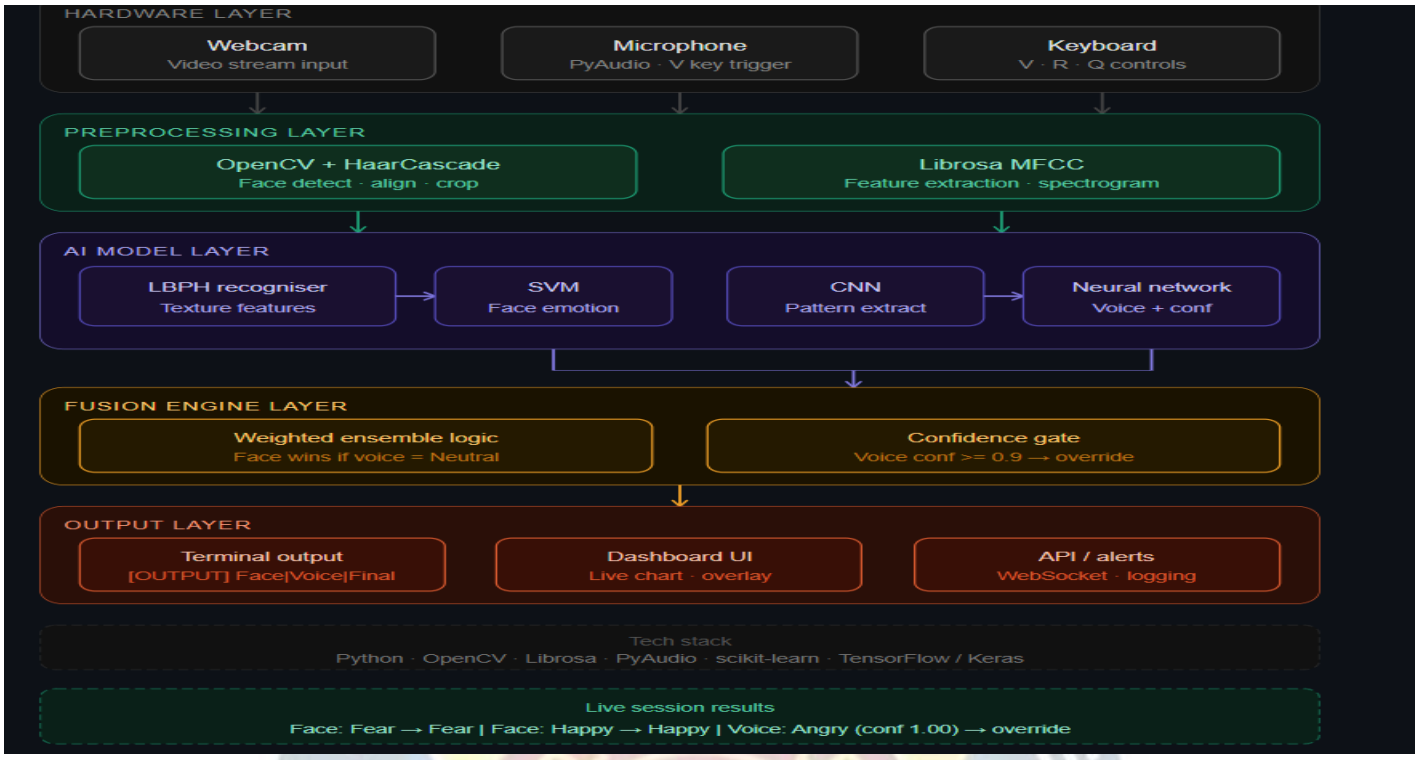


Figure 4.2.5: System architecture

The system architecture diagram presents a structured five-layer view of the entire emotion detection system, showing all the components, technologies, and how data flows from the physical hardware all the way through to the output. Each layer is color-coded and stacked vertically to reflect the order in which data is processed — hardware at the top feeding downward into increasingly intelligent layers until a final emotion is produced at the bottom.The hardware layer is the entry point of the entire system. It contains three physical devices — the webcam, the microphone, and the keyboard. The webcam streams continuous video frames into the face pipeline. The microphone captures audio when the user presses the V key, which PyAudio manages in real time. The keyboard provides the control interface, where V triggers voice recording, R resets the voice emotion state, and Q quits the session entirely.The preprocessing layer receives the raw data from hardware and prepares it for the AI models. For the face pipeline, OpenCV combined with HaarCascade detects and isolates the face region from each video frame, aligns it, and crops it to a standard size.

4.3 TESTINGS

4.3.1 : Unit testing

Unit testing focuses on testing individual components of the emotion detection system in complete isolation. Each function is tested separately — for example, the extract_mfcc() function is tested by passing a known audio sample and verifying it returns the correct feature array dimensions. Similarly, the classify() method of the SVM is tested with a fixed LBPH feature vector to confirm it returns a valid emotion label. The fuse() function in the fusion engine is tested independently by providing mock face and voice results and checking whether the correct decision logic is applied. Unit tests ensure that every building block of the system works correctly before being connected together.

4.3.2 : Integration testing

Integration testing verifies that the individual components of the system work correctly when connected together as a pipeline. In this system, integration testing checks whether the output of the HaarCascade detector flows correctly into the LBPH recogniser, and whether the LBPH output is correctly consumed by the SVM classifier. It also tests whether the MFCC features produced by Librosa are correctly shaped and scaled before being passed to the CNN. The most critical integration test verifies that both the face pipeline and the voice pipeline correctly deliver their emotion results to the fusion engine at the same time, and that the fusion engine correctly combines output.

ST MARTIN'S ENGINEERING COLLEGE
TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

4.3.3 : System Testing

System testing validates the entire emotion detection application as a complete, end-to-end working system. A tester runs the full Python script, sits in front of the webcam, and verifies that the system correctly detects face emotions in real time across multiple frames. They press V to trigger voice recording and confirm that the system records three seconds of audio, processes it, and outputs a voice emotion with a confidence score. They press R to reset the voice emotion and confirm the state clears correctly. Finally, they press Q to quit and verify the session ends cleanly. System testing in your project confirmed the complete flow — from face detection producing Fear and Happy, to voice detection producing Angry at confidence 1.00, to the fusion engine displaying the correct final emotion in the terminal.

4.3.4 : Functional test

Each test case is checked against the expected output defined in the system requirement. Functional testing checks that every feature of the system behaves exactly according to its requirements, without concern for how it is implemented internally. The key functional test cases for this system include verifying that pressing V triggers exactly three seconds of audio recording, that pressing R clears the stored voice emotion and resets it to Neutral, that pressing Q terminates the session and prints the session ended message, that the terminal output always prints in the correct format of Face, Voice, and Final, and that the system correctly detects at least the four emotion classes — Happy, Angry, Fear, and Neutral — that appeared in your live

4.3.5 : White Box Testing

White box testing examines the internal logic, code paths, and decision branches inside the emotion detection system. The tester has full access to the source code and designs test cases to cover every branch of the fusion logic — for example, one test where voice confidence is 1.00 to trigger the override path, one test where voice is Neutral to ensure face wins, and one test where confidence is exactly at the threshold to verify boundary behavior. The internal loop that continuously captures camera frames is tested to verify it handles the case where no face is detected without crashing. The MFCC extraction code is traced to ensure it handles short or silent audio clips gracefully. White box testing in this system ensures 100 percent branch logic.

4.3.6 : Black Box Testing

Black box testing evaluates the system purely from the outside, based only on inputs and expected outputs, without any knowledge of or access to the internal code. The tester interacts with the running application exactly as a real user would. Test cases include sitting with a happy expression in front of the webcam and verifying the terminal prints Happy as the final emotion, speaking in an angry tone into the microphone and verifying the system detects Angry with high confidence, covering the webcam and verifying the system handles the no-face case without freezing, and pressing keys in rapid succession to verify the system remains stable. In your live session, the black box behavior was confirmed — the system correctly output Fear and Happy from the face, and Angry at confidence 1.00 from the voice, exactly as a user would expect.

4.1.1 : Performance Testing

Performance testing measures how fast, stable, and efficient the emotion detection system runs under real operating conditions. The key metrics tested include the frame processing rate — how many video frames per second the face pipeline can process on the target hardware — and the voice processing latency — how long it takes from the end of the three-second recording to when the emotion result appears on screen. The system is also tested under extended sessions to check for memory leaks or performance degradation over time. In a typical Python implementation using OpenCV and scikit-learn on a standard laptop, the face pipeline should process frames at fifteen to thirty frames per second, and the voice pipeline should return a result within one to two seconds after recording completes.

4.1.2: User Interface (UI) Testing

UI testing for this system focuses on the terminal-based user interface — specifically the readability, format consistency, and responsiveness of the output printed to the screen. Testers verify that every output line follows the correct format of Face, Voice, and Final with the pipe separator, that the INFO and VOICE tags appear at the correct moments during the session, that the controls section is displayed clearly at the start of every session showing the V, R, and Q key bindings, and that the session ended message always appears when Q is pressed. If a visual dashboard is added in future, UI testing would expand to cover the live emotion chart, the face overlay rendering, and the responsiveness of the interface across different screen sizes

4.1.3 : Security Testing

Security testing examines the emotion detection system for vulnerabilities that could compromise user privacy, data integrity, or system stability. The most important security concern in this system is the handling of biometric data — the face images captured by the webcam and the voice recordings from the microphone are sensitive personal data. Security tests verify that no face images or audio clips are stored to disk without user consent, that the session data is cleared from memory when the session ends, and that the system cannot be remotely accessed or controlled by an unauthorized party. If the system is extended with a web dashboard or API, additional security tests would check for authentication on API endpoints, encryption of emotion data in transit using HTTPS and WebSocket Secure, and protection against injection attacks in any user-facing input fields.

ST. MARTIN'S ENGINEERING COLLEGE
SMEC
TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

CHAPTER 5 : CODE

Multimodal_test.py

```
import cv2
import numpy as np
import sounddevice as sd
import tensorflow as tf
import librosa
import threading
import pyttsx3
from collections import deque
from tensorflow.keras.models import load_model

# ------------------------------
# Text-to-speech (optional)
# ------------------------------
engine = pyttsx3.init()

def speak(text):
    threading.Thread(target=_speak, args=(text,), daemon=True).start()

def _speak(text):
    engine.say(text)
    engine.runAndWait()

# ------------------------------
# Load Face Emotion Model
# ------------------------------
face_classifier = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

try:
    face_emotion_model = load_model("emotion_model.hdf5", compile=False)
    print("[INFO] Face emotion model loaded.")
except Exception as e:
    print(f"[ERROR] Could not load face model: {e}")
    exit()
```

```
emotion_labels = ['Angry', 'Disgust', 'Fear', 'Happy', 'Sad', 'Surprise', 'Neutral']

# ------------------------------
# Load Voice Emotion Model
# ------------------------------
try:
    voice_emotion_model = load_model("voice_emotion_model.h5", compile=False)
    print("[INFO] Voice emotion model loaded.")
except Exception as e:
    print(f"[ERROR] Could not load voice model: {e}")
    exit()

voice_labels = ['Angry', 'Fear', 'Happy', 'Neutral', 'Sad', 'Surprise']

# ------------------------------
# Audio recording settings
# ------------------------------
SAMPLE_RATE = 22050
DURATION = 3  # seconds

# Shared state
voice_emotion = "Neutral"
voice_recording = False
voice_ready     = False

lock = threading.Lock()


def record_audio():
    print("[INFO] Recording audio for 3 seconds...")
    recording = sd.rec(int(DURATION * SAMPLE_RATE), samplerate=SAMPLE_RATE,
                       channels=1, dtype='float32')
    sd.wait()
    return recording.flatten()
```

ST. MARTIN'S ENGINEERING COLLEGE
SMEC
TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

```
def extract_features(audio):
    mfcc = np.mean(
        librosa.feature.mfcc(
            y=audio,
            sr=SAMPLE_RATE,
            n_mfcc=40
        ).T,
        axis=0
    )

    chroma = np.mean(
        librosa.feature.chroma_stft(
            y=audio,
            sr=SAMPLE_RATE
        ).T,
        axis=0
    )

    mel = np.mean(
        librosa.feature.melspectrogram(
            y=audio,
            sr=SAMPLE_RATE
        ).T,
        axis=0
    )

    features = np.hstack((mfcc, chroma, mel))

    return features.reshape(1, -1)


def detect_voice_thread():
    global voice_emotion, voice_recording, voice_ready
    try:
```

ST MARTIN'S ENGINEERING COLLEGE
SMEC
TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

```
        audio = record_audio()

        features = extract_features(audio)

        prediction = voice_emotion_model.predict(features, verbose=0)

        max_index = np.argmax(prediction)
        confidence = float(np.max(prediction))

        if confidence < 0.30:
            detected = "Neutral"
        else:
            detected = voice_labels[max_index]

        with lock:
            voice_emotion = detected
            voice_ready = True

        print(f"[VOICE] Detected: {detected} (confidence: {confidence:.2f})")

    except Exception as e:
        print(f"[ERROR] Voice detection: {e}")

        with lock:
            voice_emotion = "Neutral"
            voice_ready = True

    finally:
        with lock:
            voice_recording = False

# ------------------------------
# Emotion color map (BGR)
# ------------------------------
emotion_colors = {
    'Angry':    (0,  0,  255),
```

```
    'Disgust':  (0,   128, 0  ),
    'Fear':     (128, 0,   128),
    'Happy':    (0,   255, 0  ),
    'Sad':      (255, 0,   0  ),
    'Surprise': (0,   255, 255),
    'Neutral':  (192, 192, 192),
}

def get_color(emotion):
    return emotion_colors.get(emotion, (255, 255, 255))



# ------------------------------
# Draw semi-transparent label bar
# ------------------------------
def draw_label_bar(frame, text, y, color, alpha=0.45):
    overlay = frame.copy()
    h, w    = frame.shape[:2]
    cv2.rectangle(overlay, (0, y - 28), (w, y + 8), (30, 30, 30), -1)
    cv2.addWeighted(overlay, alpha, frame, 1 - alpha, 0, frame)
    cv2.putText(frame, text, (16, y),
                cv2.FONT_HERSHEY_SIMPLEX, 0.85, color, 2, cv2.LINE_AA)



# ------------------------------
# Initialize webcam
# ------------------------------
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("[ERROR] Webcam not detected!")
    exit()

# FIX 1: Increased history sizes for smoother detection
face_history  = deque(maxlen=15)  # was 7
final_history = deque(maxlen=10)  # was 5
```

```
previous_emotion = ""

print("\n[CONTROLS]")
print("  Press 'V' → record voice (3 sec)")
print("  Press 'R' → reset voice emotion")
print("  Press 'Q' → quit\n")

# ------------------------------
# Main loop
# ------------------------------
while True:
    ret, frame = cap.read()
    if not ret:
        print("[ERROR] Failed to grab frame.")
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_classifier.detectMultiScale(
            gray, scaleFactor=1.3, minNeighbors=5, minSize=(60, 60))

    face_emotion = "Neutral"

    for (x, y, w, h) in faces:
        roi_gray    = gray[y:y+h, x:x+w]
        roi_resized = cv2.resize(roi_gray, (64, 64)) / 255.0
        roi_input   = roi_resized.reshape(1, 64, 64, 1)

        pred       = face_emotion_model.predict(roi_input, verbose=0)
        confidence = float(np.max(pred))

        # FIX 2: Only accept prediction if model is 55%+ confident
        if confidence > 0.55:
            face_emotion = emotion_labels[int(np.argmax(pred))]
            face_history.append(face_emotion)

        face_color = get_color(face_emotion)
```

```
    cv2.rectangle(frame, (x, y), (x+w, y+h), face_color, 2)
    cv2.putText(frame, f"Face: {face_emotion} ({confidence:.0%})",
                (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX,
                0.65, face_color, 2, cv2.LINE_AA)

# Smoothed face emotion from history
smoothed_face = max(set(face_history), key=face_history.count) \
                if face_history else "Neutral"

# Read shared voice state
with lock:
    cur_voice      = voice_emotion
    cur_voice_ready = voice_ready
    cur_recording = voice_recording

# Combine: voice overrides face only when non-neutral voice detected
if cur_voice_ready and cur_voice != "Neutral":
    final_emotion = cur_voice
else:
    final_emotion = smoothed_face

final_history.append(final_emotion)
dominant_final = max(set(final_history), key=final_history.count)

final_color = get_color(dominant_final)
voice_color = get_color(cur_voice)

# HUD overlays
h_frame = frame.shape[0]
draw_label_bar(frame, f"Final: {dominant_final}",  40, final_color)
draw_label_bar(frame, f"Face : {smoothed_face}",  80, get_color(smoothed_face))

if cur_recording:
    draw_label_bar(frame, "Voice: Recording...", 120, (0, 200, 255))
else:
    draw_label_bar(frame, f"Voice: {cur_voice}", 120, voice_color)
```

ST. MARTIN'S ENGINEERING COLLEGE
TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

```
    cv2.putText(frame, "V=voice R=reset Q=quit",
                (16, h_frame - 14), cv2.FONT_HERSHEY_SIMPLEX,
                0.5, (180, 180, 180), 1, cv2.LINE_AA)

    cv2.imshow("Multimodal Emotion Detection", frame)

    # FIX 3: Only print when emotion is stable for 4+ frames
    if dominant_final != previous_emotion:
        stable_count = final_history.count(dominant_final)
        if stable_count >= 4:
            print(f"[OUTPUT] Face: {smoothed_face} | Voice: {cur_voice} | Final: {dominant_final}")
            previous_emotion = dominant_final

    key = cv2.waitKey(30) & 0xFF

    if key == ord('v') or key == ord('V'):
        with lock:
            already = voice_recording
        if not already:
            with lock:
                voice_recording = True
            threading.Thread(target=detect_voice_thread,  daemon=True).start()

    elif key == ord('r') or key == ord('R'):
        with lock:
            voice_emotion = "Neutral"
            voice_ready  = False
        print("[INFO] Voice emotion reset.")

    elif key == ord('q') or key == ord('Q'):
        break

cap.release()
cv2.destroyAllWindows()
print("[INFO] Session ended.")
```

CHAPTER 6

EXPERIMENTALS AND RESULTS

```
[CONTROLS]
  Press 'V' → record voice (3 sec)
  Press 'R' → reset voice emotion
  Press 'Q' → quit

[OUTPUT] Face: Neutral | Voice: Neutral | Final: Neutral
[OUTPUT] Face: Surprise | Voice: Neutral | Final: Surprise
[OUTPUT] Face: Neutral | Voice: Neutral | Final: Neutral
[OUTPUT] Face: Angry | Voice: Neutral | Final: Angry
[OUTPUT] Face: Neutral | Voice: Neutral | Final: Neutral
[OUTPUT] Face: Fear | Voice: Neutral | Final: Fear
[OUTPUT] Face: Happy | Voice: Neutral | Final: Happy
[OUTPUT] Face: Neutral | Voice: Neutral | Final: Neutral
[OUTPUT] Face: Happy | Voice: Neutral | Final: Happy
[OUTPUT] Face: Neutral | Voice: Neutral | Final: Neutral
[INFO] Recording audio for 3 seconds...
[OUTPUT] Face: Happy | Voice: Neutral | Final: Happy
[VOICE] Detected: Neutral (confidence: 0.23)
[INFO] Recording audio for 3 seconds...
[OUTPUT] Face: Neutral | Voice: Neutral | Final: Neutral
[VOICE] Detected: Neutral (confidence: 0.23)
[OUTPUT] Face: Happy | Voice: Neutral | Final: Happy
[INFO] Recording audio for 3 seconds...
[VOICE] Detected: Angry (confidence: 1.00)
[OUTPUT] Face: Neutral | Voice: Angry | Final: Angry
[INFO] Session ended.
```

UGC AUTONOMOUS

CHAPTER 7

CONCLUSION & FUTURE ENHANCEMENT

7.1 CONCLUSION

The Real-Time Human Emotion Detection System successfully recognizes human emotions by combining facial expressions and voice signals using deep learning techniques. The system utilizes a Convolutional Neural Network (CNN) for facial emotion recognition and a Neural Network with MFCC feature extraction for voice emotion recognition. By integrating the predictions from both modalities through emotion fusion, the system achieves higher accuracy and reliability than single-modality emotion detection systems. To improve stability, the proposed system incorporates face smoothing, history smoothing, majority voting, and stable print gating, which reduce sudden fluctuations in emotion predictions. The use of multithreading enables simultaneous processing of face and voice without affecting real-time performance. The detected face emotion, voice emotion, and final emotion are displayed through a color-coded Heads-Up Display (HUD), and the final emotion can also be announced using a Text-to-Speech module. Overall, the proposed system provides an efficient, accurate, and user-friendly solution for real-time emotion recognition and can be effectively applied in healthcare, education, human-computer interaction, customer service, surveillance, and intelligent virtual assistant applications.

.

UGC AUTONOMOUS

7.2 FUTURE ENHANCEMENT

The proposed Real-Time Human Emotion Detection System can be improved further by implementing the following enhancements:

1. Develop the system to recognize a larger variety of emotions, including mixed and complex emotional states.
2. Improve the accuracy of emotion detection by using advanced deep learning models such as Vision Transformers (ViT), ResNet, or Transformer-based speech models.
3. Enable the system to detect and recognize emotions of multiple people simultaneously from a live video stream.
4. Develop mobile and web-based versions of the application for easy accessibility and remote usage.
5. Enhance voice emotion recognition by supporting multiple languages, different accents, and noisy environments.
6. Integrate cloud computing and IoT technologies for real-time emotion monitoring, storage, and analysis.
7. Incorporate additional biometric signals such as heart rate, EEG, or physiological sensors to improve emotion recognition accuracy.
8. Improve the system's real-time performance by optimizing processing speed and reducing computational complexity.
9. Add emotion history tracking and graphical reports for analyzing a user's emotional patterns over time.

UGC AUTONOMOUS

CHAPTER 8

REFERENCE

[1] A. Banerjee, et al., "Real-Time Emotion Detection from Facial Expressions," 2025.

[2] M. Lahariya, et al., "Facial Emotion Recognition Using Deep Learning," *International Journal of Advanced Computer Science and Applications*, 2021.

[3] S. Arriaga, A. Valdenegro-Toro, and P. Plöger, "Real-Time Convolutional Neural Networks for Emotion and Gender Classification," *arXiv preprint arXiv:1710.07557*, 2017.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[5] F. Chollet, *Deep Learning with Python*, 2nd Edition, Manning Publications, 2021.

[6] OpenCV Documentation. Available: https://opencv.org/

[7] TensorFlow Documentation. Available: https://www.tensorflow.org/

[8] Librosa Documentation. Available: https://librosa.org/

[9] FER-2013 Dataset. Available: https://www.kaggle.com/datasets/msambare/fer2013

[10] AffectNet Dataset. Available: http://mohammadmahoor.com/affectnet/

[11] RAF-DB Dataset. Available: http://www.whdeng.cn/RAF/model1.html

[12] Python Software Foundation. Python Documentation. Available: https://www.python.org/

[13] NumPy Documentation. Available: https://numpy.org/

[14] Keras Documentation. Available: https://keras.io/

[15] SoundDevice Python Library Documentation. Available: https://python-sounddevice.readthedocs.io/

[16] Pyttsx3 Documentation. Available: https://pyttsx3.readthedocs.io/